

Modular Design

Eunbyung Park
Yonsei University



Textbook Chapter 7

Modular Design

Disclaimer: the following slides are based on the lecture slides provided by the textbook author, Copyright 2013 John Wiley and Sons. Material added/revised by the course instructor Copyright 2024, Yonsei University.

These course materials have been produced and published for the purpose of teaching Yonsei University students, so they cannot be used for any other purposes nor be shared with others. The legal responsibility for the violation lies with the actor him/herself.

Chapter 7 Modular Design

Until now, we have looked at programs comprised of a set of individual functions. In complex software systems, however, programs are organized at a higher level as a set of modules, each module containing a set of functions (or objects). Modules, like functions, are a fundamental building block in software development.

Modules are an extension of the following concepts of software development that you already know from our discussion of functions:

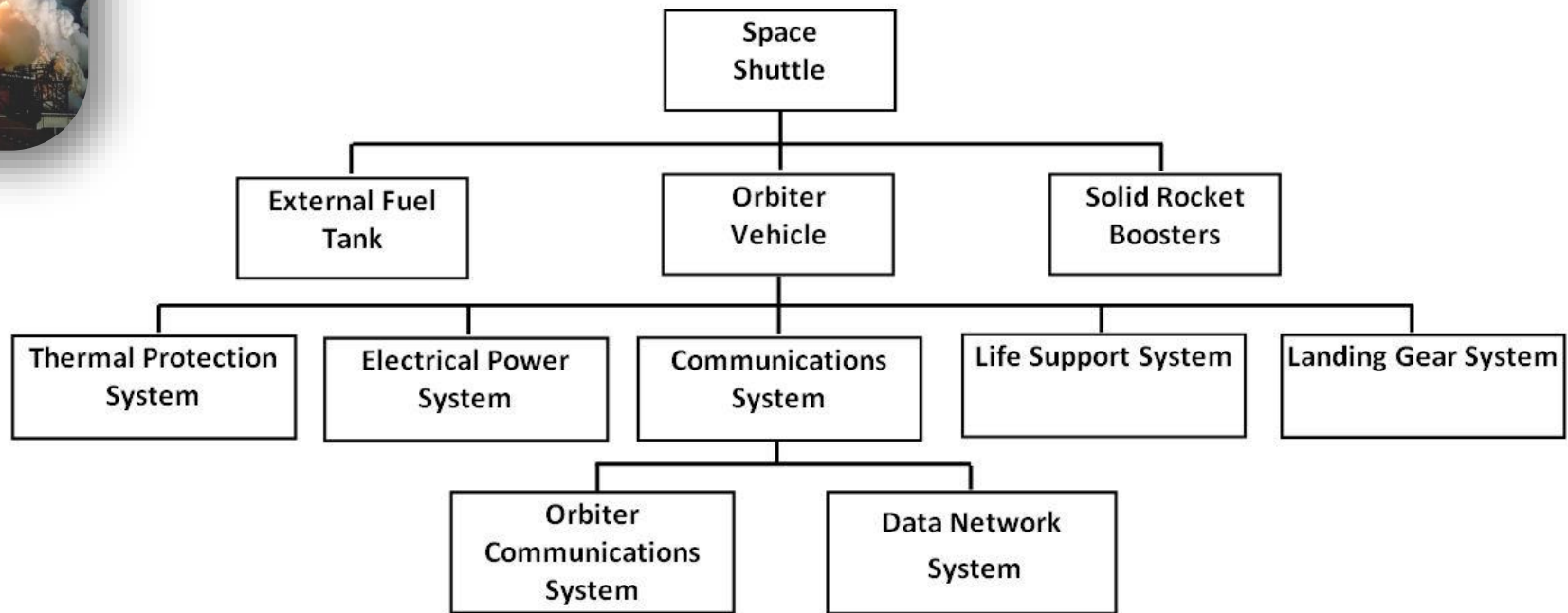
- 1) decomposition*
- 2) abstraction*
- 3) re-use*

Motivation

- Software systems are some of the most complex entities ever created.
- Web browsers and operating systems, for example, contain as many as 50–100 million lines of code, and take as long as 10,000 person-years to develop.
- We can make complex systems more manageable by designing them as a set of subsystems, or modules.
- For example, NASA's space shuttle is one of the most complex systems ever engineered, containing more than 2.5 million parts.
 - The major components are the orbiter vehicle, a large external liquid-fuel tank, and two solid rocket boosters.
 - The orbiter vehicle is composed of several subsystems (e.g., the communications system), which in turn may be composed of sub-subsystems (e.g., the data network system).
- Although many involved with the shuttle may understand its overall design, only a few individuals need to understand or be involved with the detailed design, implementation, and testing of any specific subsystem.
- The same is true for the modular design of software. In this chapter, we look at the issue of modular program design.



- Modules
- Hierarchy between modules



Modular Design of the NASA Space Shuttle

Modules

We look at the important notion of a **module** in software development.

What Is a Module?

Module:

- the design and/or implementation of specific functionality to be incorporated into a program.
- Modules consist of a collection of functions.
- Example: the Python turtle module
- The use of modules has a number of advantages.
 - See next slide.

Advantages of Modules

- 1) **Decompose** large programs into **manageable-sized** parts
- 2) Each part (module) is assigned a clearly specified functionality.
- 3) **Separating programming tasks** between programmers or teams.
- 4) Allows modules to be **individually developed** and **tested**, and eventually integrated as a part of a complete system.
- 5) Modular design **facilitates program maintenance**.

The code responsible for a given feature of the software is **localized** in a small number (preferably 1) of modules, and not sprinkled across various parts of the program.

- **Software Design**
 - Building complex systems from smaller subsystems (composability of modules).
- **Software Development**
 - Provides a means of **dividing up** programming tasks
 - Provides a means for the **reuse** of program code.
- **Software Testing**
 - Provides a means of **separately testing** parts of a program.
 - Provides a means of **integrating parts** of a program during testing.
- **Software Modification and Maintenance**
 - Facilitates the **modification of specific** program functionalities.

Benefits of Modular Software Design

Real-world Example: Modules in the Mozilla Project

- <https://www.mozilla.org/en-US/about/governance/policies/module-ownership/>
- The Mozilla project is far too big for any one person [...]
- Instead, decision-making is distributed to a range of participants through the project being split into **modules**.
- Each module has a **module owner**.
 - A module owner is the person to whom leadership of a module's work has been delegated (**module ownership**).
- A number of larger modules (e.g., Firefox, Thunderbird, SpiderMonkey, Bugzilla) have sub-modules subordinate to them.

Standard Library Modules

- All programming languages come with a **standard library** of functionality to ease the life of programmers.
- Some functionality is built into the core of the language.
 - Examples in Python: built-in functions, operators, and objects and their methods: `print()`, `input()`, `min()`, `del`, `range()`, lists, tuples, ...
- More specialized functionality is packaged separately as additional files.
 - In Python those files are called **modules**.
- Python modules must be requested by the programmer through the **import statement**.
 - `import math`
 - `import turtle`
 - `import os`

Modules

- Modules are containers for functions and code.
- A module bundles together related functionality.
 - Only if the programmer imports the module, such specific functionality will be loaded (and consume memory).
- Across different programming languages, the terms ``libraries" and ``packages" are other names for modules.
- Java and Python are famous for their rich eco-system of libraries, which contributed to the success of theses languages.
- A program which is using a module is called a **client** of that module.
 - **Example:** in previous labs, your code used the Turtle module.
→ Your code is a **client** of the Turtle module.

Using Modules

- Once a module is loaded, we use the dot operator to use functions and constants from the module:
 - `turtle.penup()` # function call
 - `math.sqrt(n)` # function call
 - `math.pi` # constant 3.141592...

What Is a Python Module?

- A **Python module** is a **.py** file which contains
 - Python definitions
 - Variables
 - Functions
 - Python statements

Circle Example Module

```
pi = 3.14

def area(radius):
    return (radius**2) * pi

def circumference(radius):
    return 2 * pi * radius

def sphereSurface(radius):
    return 4.0 * area(radius)
```

module circle.py

import module

```
import circle
```

```
pi = 3
```

dot notation

```
print(pi, circle.pi )
```

```
print(circle.area(3))
```

```
x=circle.circumference(3)
```

```
y=circle.sphereSurface(4)
```

```
print(x, y)
```

main program (module client)

- The import statement makes the definitions from the module visible to the main program.
- **Dot notation** used to refer to names in the module:
 - `module_name.definition_name`

Module Namespace Example

```
pi = 3.14
```

*pi in module's
namespace*

```
def area(radius):  
    return (radius**2) * pi
```

```
def circumference(radius):  
    return 2*pi*radius
```

```
def sphereSurface(radius):  
    return 4.0 * area(radius)
```

module circle.py

```
import circle
```

```
pi = 3
```

```
print(pi, circle.pi )
```

```
print(circle.area(3))
```

```
x=circle.circumference(3)
```

```
y=circle.sphereSurface(4)
```

```
print(x, y)
```

*pi from main
program's namespace
pi from module's
namespace*

main program (module client)

- A module has its own **namespace**.
- In the main program, the dot notation allows us to refer to names in the namespace of the module.
 - **circle.pi** (3.14) versus **pi** (3)
 - Output of first print statement: 3 3.14

Statements Inside Modules

```
def func1():  
    print('func1 called')  
  
def func2():  
    print('func2 called')
```

```
print('Module "simple" loaded!')  
pi = 3.14
```

*statements
inside module*

module simple.py

```
import simple
```

```
simple.func1()
```

```
simple.func2()
```

main program

Output:

```
Module "simple" loaded!
```

```
func1 called
```

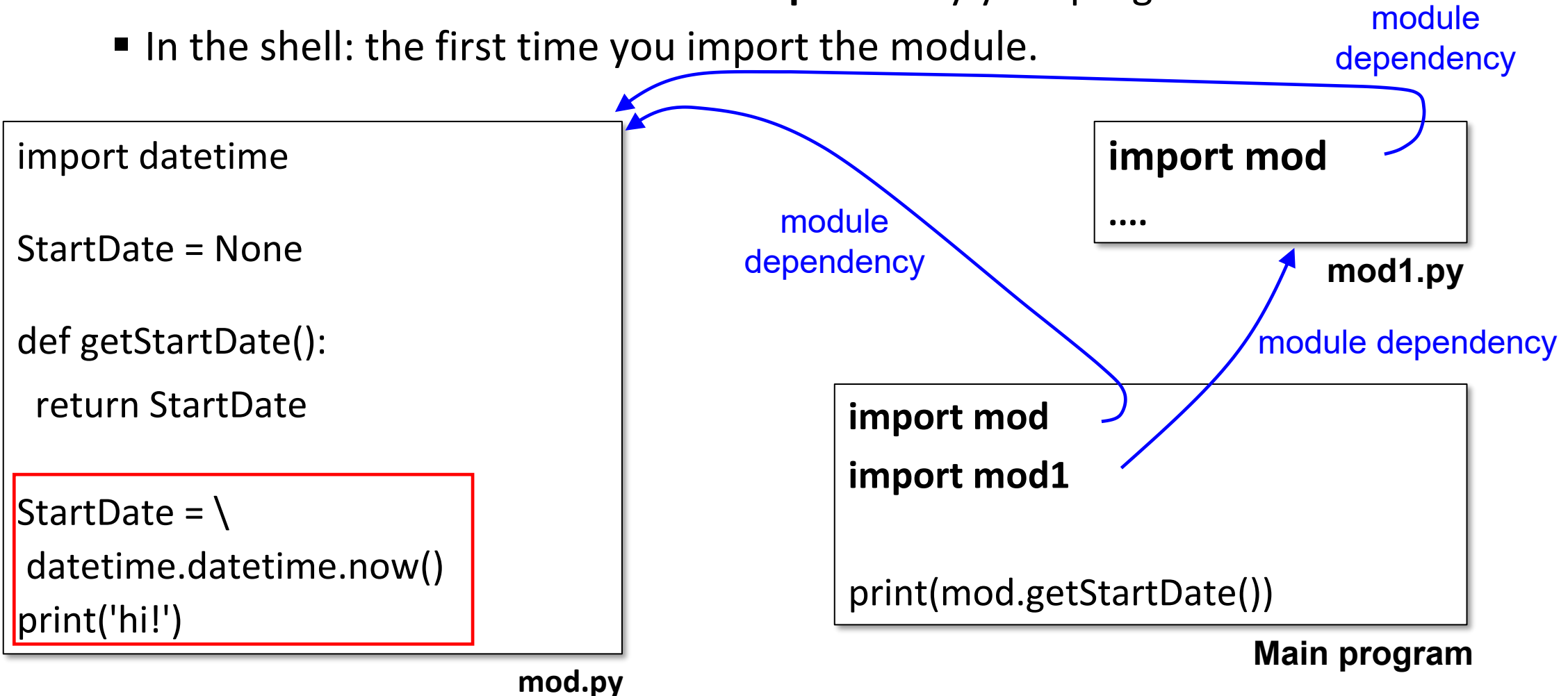
```
func2 called
```

- A Python module may contain **statements**.
 - Similar to the main program of the module's client.
- Such statements are executed when the module is imported.
- **Purpose:** initialize the module (if required).
 - Above example: pi initialized to 3.14.

Python Module Initialization

Modules may contain a set of statements to **initialize** the module.

- Those are executed **exactly once per program run**.
 - The **first time** that the module is **imported** by your program.
 - In the shell: the first time you import the module.



__main__: the name of the main module

```
def func1():  
    print('func1 called')  
  
if __name__ == "__main__":  
    print('Running as the main module.')  
    func1()  
else:  
    print('Running as an imported module.')  
    print('My name is', __name__)
```

less_simple.py

```
import less_simple  
  
print('Less_simple_client')
```

less_simple_client.py

During program execution, for each module Python **defines** a `__name__` variable.

- For the **main module**, the name will be `"__main__"`.
 - The main module is the file that contains your main program.
- This mechanism can be used to determine whether a file is executed as the main module or as an imported module.
- When **less_simple_client.py** is executed, the output will be:
Running as an imported module.
My name is less_simple
Less_simple_client

Where are Python Modules Located?

```
import math
```

```
...
```

- import looks for a file with the given name and the **.py** extension (ex.: math.py).
- First it looks in the "local" directory (the same directory where your main program's file lives).
- If it finds a file with the name of the module it's looking for, it will assume this is the module you want.
- If you're trying to use the turtle module, don't have a file "turtle.py" in your local directory! (See next slide.)
- If it can't find the module in the local directory, it looks in directories listed in the PYTHONPATH environment variable (if set).
- Then it looks in a Python installation-specific directory (e.g., C:\Python64\Lib)
- If still not found, an error is reported and the program terminates.

Locating Your Modules from the PyCharm Console

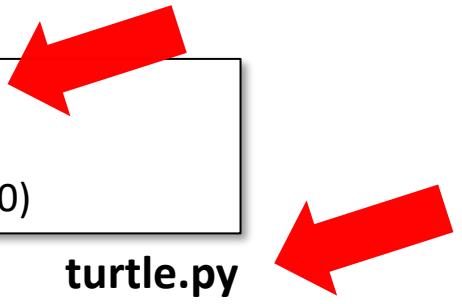
```
>>> import circle
Traceback (most recent call last):
  File "<input>", line 1, in <module>
...
module = self._system_import(name, *args, **kwargs)
UserssModuleNotFoundError: No module named 'circle'

>>> import sys
>>> sys.path.append('C:\\Users\\bburg\\Desktop\\circle')
>>> import circle
>>> circle.pi
3.14
```

- Use the sys module to append the directory with your module to the path that sys searches for modules.

An infamous mistake

```
import turtle  
turtle.setup(650, 620)
```



turtle.py

```
Traceback (most recent call last):  
  File "turtle.py", line 1, in <module>  
    import turtle  
  File "/home/bburg/win7/repos/CPugrad_ownsbn/2017/CPu17_slides/src/ch7/turtle.py",  
    line 3, in <module> turtle.setup(650, 620)  
AttributeError: module 'turtle' has no attribute 'setup'
```

- If a programmer accidentally names the main module the same as another module, there is a **name clash** of module names.
- Python does not detect this name clash.
 - The Python module mechanism is **weak**.
- Rather, Python imports the main module into itself.
- This import shadows the other module.
 - The "real" turtle module in the above example.
 - The "real" turtle module's functions are not imported → error

Naming convention:

Python modules are named using lower case letters and optional underscore characters.

Examples:

- turtle
- math
- big_data_analysis

Namespaces of Functions and Modules

- A **namespace** is a container that **provides a named context** for a set of identifiers.
- Namespaces enable programs to avoid **name clashes** by associating each identifier with the namespace from which it originates.
 - For an example of a name clash, see the following slide.

- A function body is a namespace:

Local variables from the function body are **not visible** anywhere else.

```
def doSomething( x, y ):
```

```
    SomeVar = 0
```

```
    ...
```

Namespace of function doSomething

- Contains variables x, y, SomeVar
- Those variables are not visible outside of the function body.

- The main program (global frame) is another namespace.
- Each module is a namespace:
 - Module's global variables and functions **not externally visible**...
 - ...unless the module is imported.


```
# module1:
```

```
def double (lst):
```

```
    """ Returns a new list  
        with each number  
        doubled.
```

```
    [1, 2, 3] becomes
```

```
    [2, 4, 6]
```

```
    """
```

```
    ...
```

```
module1.py
```

```
# module2:
```

```
def double (lst):
```

```
    """ Returns a new list  
        with each number  
        doubled.
```

```
    [1, 2] becomes
```

```
    [(1,1),(2,2)]
```

```
    """
```

```
    ...
```

```
module2.py
```

**Ambiguous
reference to
`double`**

```
import module1
```

```
import module2
```

```
# main:
```

```
double([1, 3, 5])
```

Case study (not what Python is doing):

- If Python would import function names directly into the client, this example would introduce a **name clash** for ``double``.
- We cannot import one name (``double``) from two modules at the same time.

Fully Qualified Names

- In Python, each module has its own **namespace**.
- Contains the names of all items in the module:
 - 1) **Names of functions from the module**
 - 2) **Names of global variables from the module**
- The two instances of **identifier double**, from the previous slide, are **disambiguated** by their **fully qualified** names:
 - **module1.double** **# fully qualified name 1**
 - **module2.double** **# fully qualified name 2**
- See the following slide.

Fully Qualified Function Names

```
import module1  
import module2
```

```
# main:
```

```
ans1 = module1.double([1, 3, 5])
```

```
ans2 = module2.double([33, 15, 10])
```

References function ``double''
from namespace of module1.

References function ``double''
from namespace of module2.

`from-import' Declarations

```
from math import floor, ceil
```

```
x = floor(3.5)
```

```
y = ceil(3.5)
```

- Imports the named functions (floor, ceil) into the program's namespace.
 - No need for fully-qualified name.
 - Example: floor(3.5) instead of math.floor(3.5)
- Can be used to **resolve name clashes**:

```
from math import floor as fl
```

```
x = fl(3.5)
```

```
floor = 10 # use name ``floor'' for variable
```

In this name space, the floor function from math is renamed to fl.

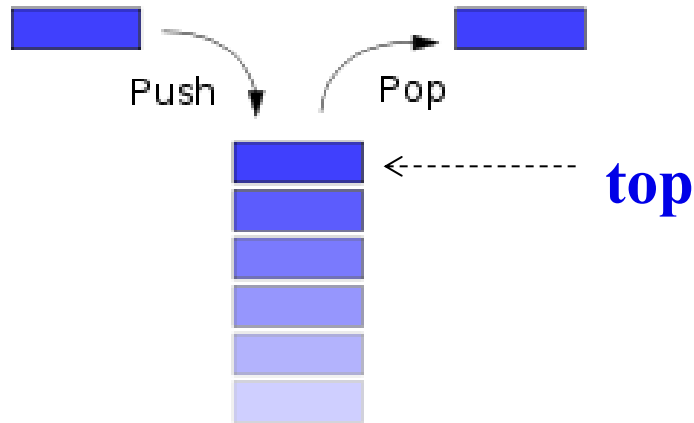


A Programmer-defined Module

- To demonstrate the development of a programmer-defined module, we present an example stack module.
- A stack is a very useful data structure in computer science.
- Stacks are used to temporarily store and retrieve data.
- They have the property that the last item placed on the stack is the first to be retrieved.
- This is called last-in-first-out (LIFO).
- A stack can be viewed as a list that can be accessed only from one side.

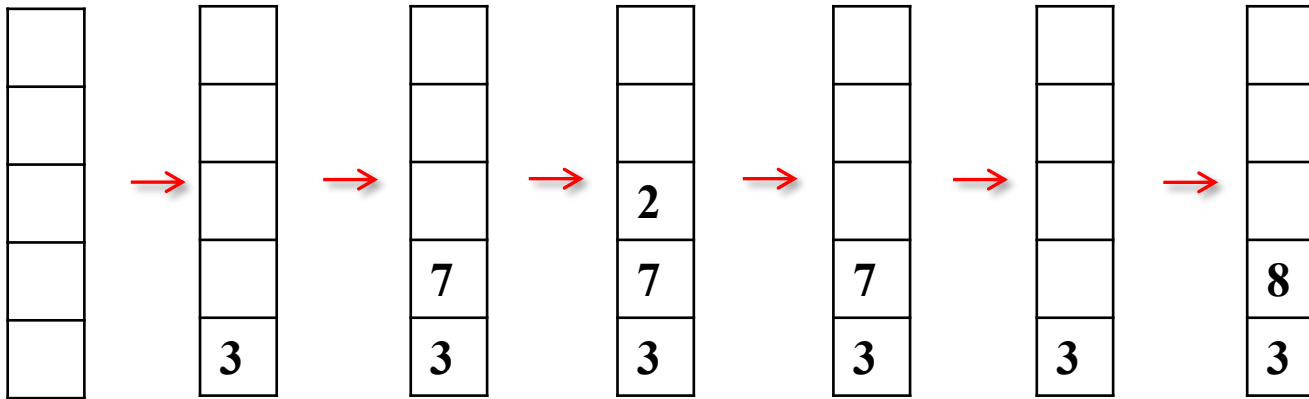
Example: Stacks

- A **stack** is a data structure with two operations:
 - **Push** an item (add to the stack top)
 - **Pop** an item (remove it from the stack top)
- **top**: position of the top-of-the-stack



Stacks have **last-in-first-out** semantics:

push(3) # top position = 1
push(7) # top position = 2
push(2) # top position = 3
val = pop() # top position = 2, val = 2
val = pop() # top position = 1, val = 7
push(8) # top position = 2



Pushing onto a full stack or popping from an empty stack is **undefined**

→ program needs to handle this case!

- See stack implementation on following slides.

- Following is a Python module containing a set of functions that implement the stack behavior.
- The stack module consists of five functions—**getStack**, **isEmpty**, **top**, **push**, and **pop**.
- The stack is **implemented as a list**.
- Only the last element in the list is accessed—that is where all items are “pushed” and “popped” from.
- Thus, the end of the list logically functions as the “top” of stack.

Stack Module (part 1)

```
def getStack():
```

```
    '''Create and return an empty stack.'''
```

```
    return []
```

```
def isEmpty(s):
```

```
    '''Return True if stack empty, otherwise False.'''
```

```
    if s == []:
```

```
        return True
```

```
    else:
```

```
        return False
```

```
def top(s):
```

```
    '''Return the top item of stack, item not removed.
```

```
    Otherwise, return None.'''
```

```
    if isEmpty(s):
```

```
        return None
```

```
    else:
```

```
        return s[len(s) - 1]
```

Stack Module (part 2)

```
def push(s,item):
```

```
    """Push item onto the top of stack."""
```

```
    s.append(item)
```

```
def pop(s):
```

```
    """
```

```
    Remove and return top item of stack.
```

```
    If stack empty, return None.
```

```
    """
```

```
    if isEmpty(s):
```

```
        return None
```

```
    else:
```

```
        item = s[len(s) - 1]
```

```
        del s[len(s) - 1]
```

```
        return item
```

Use of the Stack Module

```
import stack
```

```
mystack = stack.getStack()
```

```
for item in range(1,5):  
    stack.push(mystack, item)  
    print('Pushing', item)
```

```
while not stack.isEmpty(mystack):  
    item = stack.pop(mystack)  
    print('Popping', item)
```

client

Output:

Pushing 1

Pushing 2

Pushing 3

Pushing 4

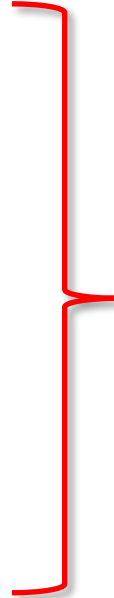
Popping 4

Popping 3

Popping 2

Popping 1

**LIFO
order**



- A new stack is created by assigning variable mystack the result of the call to function getStack().
- Values 1 – 4 are pushed onto the stack.
- As long as the stack is not empty, values are popped.
 - In last-in-first-out (LIFO) order

Breaking the Abstraction (not a good thing ☹️)

- In the **right-hand side example**, the programmer exploited the knowledge that **stacks** are **internally** represented as **lists**.

- Directly applied `append()` method on ```stack``` (read: list) to save some typing. ☹️.

```
import stack

st = stack.getStack()
```

```
for item in range(1,5):
    stack.push(st, item)
```

```
while not stack.isEmpty(st):
    item = stack.pop(st)
    print(item)
```

client 1



```
import stack

st = stack.getStack()
```

```
for item in range(1,5):
    st.append(item)
```

```
while not stack.isEmpty(st):
    item = stack.pop(st)
    print(item)
```

client 2

Breaks stack abstraction

- This is **bad programming practice**.
 - It breaks the encapsulation (```privacy```) of the stack module.
 - If the stack module is changed to reverse the order (push/pop from **front** of list), this code will **break**.
- You should only use functions from the **stack module** with variable `st`.

Use of Procedural-style Stack Implementation

```
from stack import getStack,  
isEmpty, pop, push
```

```
st = getStack()
```

Function
call to
create stack

```
for item in range(1,5):  
    push(st, item)
```

Function
call

```
while not isEmpty(st):  
    item = pop(st)  
    print(item)
```

Function
call

- Stack implementation using functions in module **stack**.
- Stack-related functions imported from the **stack** module.
- A stack is created by calling function **getStack()**.
- The stack is manipulated (mutated) through function calls.
- Must provide stack object (st) **as an argument** with each call.

Use of OOP-style Stack Implementation

```
from oop_stack import Stack
```

```
st = Stack()
```

Create
object

```
for item in range(1,5):  
    st.push(item)
```

Method
call

```
while not st.isEmpty():  
    item = st.pop()  
    print(item)
```

Method
call

- Stack implementation by object-oriented programming (OOP).
- Class **Stack** imported from module **oop_stack**.
 - After creating a **Stack** object (instantiation), the object is manipulated (mutated) through **method calls**.
 - Not new. We use method calls to manipulate list objects, tuples, etc. 😊
- The OOP interface allows us to call a method **on the stack object** (avoids stack argument).

- An object consists of data and methods.
- Only the methods provided by the object can be used to access and change the data.
- Ex.: With the OOP stack implementation, **only the methods provided by the stack object (push/pop, ...) can be used to manipulate the stack.**
 - The abstraction breach from Slide 38 can thereby be avoided.
- Called **encapsulation** (a fundamental advantage of OOP).
- It's therefore better to implement the stack as an object.
 - All of Python's data types are implemented as objects (integers, floats, lists, ...)
 - Remember: everything in Python is an object.
- We will see how to do this in detail in Chapter 10 of the course.

Self-Test Questions

1. Any initialization code in a given Python module is only executed once, the first time that the module is loaded. (True/False)
2. With the “import moduleName” form of import, any utilized entities from the imported module must be prefixed with the module name. (True/False)
3. By convention, variables names in a given module beginning with the _____ character are meant to be treated as private variables of the module.
4. When importing modules, all Python Standard Library modules must be imported before any programmer-defined modules, otherwise a runtime error will occur. (True/False)
5. If a given module is imported more than once into a Python program, the Python interpreter will ensure that the module is only loaded and executed the first time that it is imported. (True/False)
6. The _____ command can be used to force the reloading of a given module, useful for when working interactively with the Python interpreter.
7. The three active namespaces that can occur during the execution of any given Python program are the _____ and _____ namespaces.

ANSWERS: